Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Розробка багатовіконного інтерфейсу користувача для графічного редактора об’єктів»

Виконав:

студент групи ІМ-о51
Мосендз Белла Яківна
Номер в списку - 9

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab5.

2. Написати вихідний текст програми згідно варіанту завдання.

3. Скомпілювати вихідний текст і отримати виконуваний файл програми.

4. Перевірити роботу програми. За потреби налагодити програму.

5. Проаналізувати та прокоментувати результати та вихідний текст програми.

6. Оформити звіт.

Варіанти завдань та основні вимоги

1. Номер варіанту завдання дорівнює номеру зі списку студентів у журналі. Студенти з непарним номером (1, 3, 5, . . .) програмують глобальний статичний об'єкт класу MyEditor у вигляді Singleton Меєрса.

2. Усі кольори та стилі геометричних форм – як у попередній лабораторній роботі №4.

3. Запрограмувати вікно таблиці. Для його відкриття та закриття передбачити окремий пункт меню. Вікно таблиці повинно автоматично закриватися при виході з програми.

4. Вікно таблиці – немодальне вікно діалогу. Таблиця повинна бути запрограмована як клас у окремому модулі. Інтерфейс модуля у вигляді оголошення класу таблиці

5. Запрограмувати запис файлу множини об’єктів, що вводяться

6. Ієрархія класів та побудова модулів повинні бути зручними для можливостей додавання нових типів об'єктів без переписування коду вже існуючих модулів.

7. У звіті повинна бути схема успадкування класів – діаграма класів. Для побудови діаграми класів дозволяється обрати інструменти на власний розсуд.

8. Бонуси-заохочення, які можуть суттєво підвищити оцінку лабораторної роботи. Оцінка підвищується за виконання кожного пункту, з наведених

нижче:

1) Якщо у вікні таблиці буде передбачено, щоб користувач міг виділити курсором рядок таблиці і відповідний об’єкт буде якось виділятися на зображенні у головному вікні.

2) Якщо у вікні таблиці користувач може виділити курсором рядок таблиці і відповідний об’єкт буде вилучено з масиву об’єктів.

3) Якщо програма не тільки записує у файл опис множини об’єктів, а ще й здатна завантажити такий файл і відобразити відповідні об’єкти у головному вікні та вікні таблиці.

При виконанні бонусів 1 та 2 забороняється робити для цього нові залежності модуля my_table від інших .xxx файлів. Тоді як надіслати повідомлення (наприклад, про виділення користувачем якогось рядка таблиці) від вікна таблиці клієнту цього вікна (наприклад, коду головного файлу)? Підказки можна знайти у матеріалі лекції стосовно технології Callback, а також патернів Observer, Listener

Main file: MainWindow.xaml.cs

```
using System;

using System.Collections.Generic;

using System.Windows;

using System.Windows.Controls;

using System.Windows.Input;

using Microsoft.Win32;

using System.IO;

using System.Text.Json;



namespace Lab5;



public partial class MainWindow : Window

{

    private MyTableWindow? _tableWindow;

    public MainWindow()

    {

        InitializeComponent();

        DrawToolbarIcons();

        App.Editor.AttachCanvas(drawingCanvas);

        App.Editor.ShapesChanged += Editor_ShapesChanged;

        SelectTool(DrawingTool.Point);
```

```
        UpdateShapeCount();
    }

    private void Editor_ShapesChanged()
    {
        UpdateShapeCount();
        RefreshTable();
    }

    private void SelectTool(DrawingTool tool)
    {
        MyEditor.Instance.SelectTool(tool);

        pointMenuItem.IsChecked =
            tool == DrawingTool.Point;

        lineMenuItem.IsChecked =
            tool == DrawingTool.Line;

        rectangleMenuItem.IsChecked =
            tool == DrawingTool.Rectangle;
```

```
        ellipseMenuItem.IsChecked =
            tool == DrawingTool.Ellipse;

        lineWithCirclesMenuItem.IsChecked =
            tool == DrawingTool.LineWithCircles;

        cubeMenuItem.IsChecked =
            tool == DrawingTool.Cube;
    }

    private void OnNotify(object sender, RoutedEventArgs e)
    {
        FrameworkElement element = (FrameworkElement)sender;
        string toolName = (string)element.Tag;
        DrawingTool tool = Enum.Parse<DrawingTool>(toolName);

        SelectTool(tool);
    }

    // масив кортежів (tuple)
```

```
    private void DrawToolbarIcons()

    {

        var icons = new (Canvas canvas, DrawingTool tool, Point start,
Point end)[]

        {

        (pointIconCanvas, DrawingTool.Point,

            new Point(20, 16), new Point(20, 16)),



        (lineIconCanvas, DrawingTool.Line,

            new Point(6, 24), new Point(34, 8)),



        (rectangleIconCanvas, DrawingTool.Rectangle,

            new Point(8, 8), new Point(32, 24)),



        (ellipseIconCanvas, DrawingTool.Ellipse,

            new Point(20, 16), new Point(32, 24)),



        (lineWithCirclesIconCanvas, DrawingTool.LineWithCircles,

            new Point(11, 16), new Point(29, 16)),



        (cubeIconCanvas, DrawingTool.Cube,

            new Point(8, 12), new Point(26, 30))
```

```
        };

        foreach (var (canvas, tool, start, end) in icons)
        {
            canvas.Children.Add(
                ShapeFactory.Create(tool, start, end).Show(false)
            );
        }
    }

    private void ToggleTable_Click(object sender, RoutedEventArgs e)
    {
        if (_tableWindow != null)
        {
            _tableWindow.Close();
            return;
        }

        var table = new MyTableWindow
        {
            Owner = this
```

```
        };


        table.RowSelected += id =>
            App.Editor.SelectFigure(id);


        table.DeleteRequested += id =>
            App.Editor.DeleteFigure(id);


        table.Closed += (_, _) =>
        {
            _tableWindow = null;
            tableMenuItem.IsChecked = false;

            App.Editor.SelectFigure(null);
        };


        _tableWindow = table;


        RefreshTable();


        table.Show();
```

```
        tableMenuItem.IsChecked = true;
    }

    private void RefreshTable()
    {
        if (_tableWindow == null)
        {
            return;
        }

        var rows = new List<TableRow>();

        foreach (FigureData figure in App.Editor.GetFigureData())
        {
            rows.Add(new TableRow
            {
                Id = figure.Id,

                Type = GetFigureName(figure.Type),
```

```
                X1 = figure.StartX,
                Y1 = figure.StartY,

                X2 = figure.EndX,
                Y2 = figure.EndY
            });
        }

        _tableWindow.SetRows(rows);
    }

    private static string GetFigureName(string type)
    {
        return type switch
        {
            "Point" => "Крапка",
            "Line" => "Лінія",
            "Rectangle" => "Прямокутник",
            "Ellipse" => "Еліпс",
            "LineWithCircles" => "Лінія з кружечками",
            "Cube" => "Каркас куба",
```

```
            _ => type
        };
    }
    private void ClearCanvas_Click(object sender, RoutedEventArgs e)
    {
        App.Editor.Clear();
    }
    private void SaveFile_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new SaveFileDialog
        {
            Filter = "JSON files (*.json)|*.json",
            DefaultExt = ".json",
            FileName = "figures.json"
        };

        if (dialog.ShowDialog(this) != true)
        {
            return;
        }
```

```
        try
        {
            FigureStorage.Save(
                dialog.FileName,
                App.Editor.GetFigureData()
            );
        }
        catch (Exception ex) when (
            ex is IOException or UnauthorizedAccessException)
        {
            MessageBox.Show(
                $"Не вдалося зберегти файл: {ex.Message}");
        }
    }
    private void OpenFile_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog
        {
            Filter = "JSON files (*.json)|*.json"
        };
```

```
        if (dialog.ShowDialog(this) != true)
        {
            return;
        }

        try
        {
            List<FigureData> figures =
                FigureStorage.Load(dialog.FileName);

            App.Editor.LoadFigures(figures);
        }
        catch (Exception ex) when (
            ex is IOException or
                  UnauthorizedAccessException or
                  JsonException or
                  ArgumentException or
                  InvalidOperationException)
        {
            MessageBox.Show(
                $"Не вдалося відкрити файл: {ex.Message}");
```

```
        }

    }



    private void DrawingCanvas_MouseLeftButtonDown(object sender,
MouseButtonEventArgs e)

    {

        Point position = e.GetPosition(drawingCanvas);

        bool success = App.Editor.BeginDrawing(position);



        if (!success)

        {

            MessageBox.Show(

                $"Досягнуто максимальної кількості фігур
({MyEditor.MaxShapes})."

            );

        }

    }



    private void DrawingCanvas_MouseMove(object sender, MouseEventArgs e)

    {

        Point position = e.GetPosition(drawingCanvas);

        App.Editor.UpdateDrawing(position);
```

```
    }


    private void DrawingCanvas_MouseLeftButtonUp(object sender,
MouseButtonEventArgs e)
    {
        Point position = e.GetPosition(drawingCanvas);
        App.Editor.EndDrawing(position);
    }


    private void UpdateShapeCount()
    {
        shapeCountText.Text = $"Фігур: {App.Editor.ShapeCount} /
{MyEditor.MaxShapes}";
    }
}
```

DrawingTools.cs

```
namespace Lab4;


public enum DrawingTool
{
    Point,
```

```
    Line,
    Rectangle,
    Ellipse,
    LineWithCircles,
    Cube
}
```

App.xaml.cs

```
using System.Windows;


namespace Lab5;
public partial class App : Application
{
    internal static MyEditor Editor => MyEditor.Instance;
}
```

MyEditor.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Windows;
using System.Windows.Controls;
```

```
using System.Windows.Media;

using System.Windows.Media.Effects;

using Lab5.Models;



namespace Lab5;



internal sealed class MyEditor

{

    // Singleton Meepca

    private MyEditor()

    {

    }



    private static class InstanceHolder

    {

        static InstanceHolder()

        {

        }



        internal static readonly MyEditor Value = new MyEditor();

    }
```

```
internal static MyEditor Instance => InstanceHolder.Value;



internal const int MaxShapes = 110;


private sealed record FigureEntry(
    int Id,
    DrawingTool Tool,
    Shape Model,
    UIElement Visual
);


private readonly FigureEntry?[] _shapes = new FigureEntry?[MaxShapes];


private int _shapeCount;
private int _nextId = 1;


private DrawingTool _currentTool = DrawingTool.Point;


private Point? _dragStart;
```

```
    private UIElement? _previewShape;

    private Canvas? _canvas;



    private UIElement? _selectedVisual;

    private Effect? _originalEffect;



    internal int ShapeCount => _shapeCount;



    internal event Action? ShapesChanged;




    internal void AttachCanvas(Canvas canvas)

    {

        _canvas = canvas;

    }



    private Canvas GetCanvas()

    {

        return _canvas

            ?? throw new InvalidOperationException(

                "Полотно ще не підключене.");
```

```
    }



    internal void SelectTool(DrawingTool tool)
    {
        CancelDrawing();
        _currentTool = tool;
    }



    internal bool BeginDrawing(Point position)
    {
        if (_shapeCount >= MaxShapes)
        {
            return false;
        }

        if (_currentTool == DrawingTool.Point)
        {
            Shape point = ShapeFactory.Create(
                _currentTool,
```

```
                position,
                position
            );

            AddShape(point, _currentTool);
            return true;
        }

        _dragStart = position;
        GetCanvas().CaptureMouse();

        return true;
    }

    internal void UpdateDrawing(Point position)
    {
        if (_dragStart is not Point start)
        {
            return;
        }
```

```
        Canvas canvas = GetCanvas();


        RemovePreview();


        Shape shape = ShapeFactory.Create(
            _currentTool,
            start,
            position
        );


        _previewShape = shape.Show(true);


        canvas.Children.Add(_previewShape);
    }


    internal void EndDrawing(Point position)
    {
        if (_dragStart is not Point start)
        {
            return;
        }
```

```
        Canvas canvas = GetCanvas();


        RemovePreview();


        if ((position - start).Length >= 2)
        {
            Shape shape = ShapeFactory.Create(
                _currentTool,
                start,
                position
            );


            AddShape(shape, _currentTool);
        }


        _dragStart = null;


        if (canvas.IsMouseCaptured)
        {
            canvas.ReleaseMouseCapture();
```

```
        }
    }

    private void AddShape(Shape shape, DrawingTool tool)
    {
        if (_shapeCount >= MaxShapes)
        {
            return;
        }

        UIElement visual = shape.Show(false);

        _shapes[_shapeCount] = new FigureEntry(
            _nextId++,
            tool,
            shape,
            visual
        );

        _shapeCount++;
```

```
        GetCanvas().Children.Add(visual);


        ShapesChanged?.Invoke();
    }


    private void CancelDrawing()
    {
        Canvas canvas = GetCanvas();


        RemovePreview();


        _dragStart = null;


        if (canvas.IsMouseCaptured)
        {
            canvas.ReleaseMouseCapture();
        }
    }


    private void RemovePreview()
    {
```

```
        if (_previewShape == null)
        {
            return;
        }

        GetCanvas().Children.Remove(_previewShape);
        _previewShape = null;
    }

    internal List<FigureData> GetFigureData()
    {
        var result = new List<FigureData>();

        for (int i = 0; i < _shapeCount; i++)
        {
            FigureEntry entry = _shapes[i]!;

            result.Add(new FigureData
            {
                Id = entry.Id,
                Type = entry.Tool.ToString(),
```

```
            StartX = entry.Model.Start.X,
            StartY = entry.Model.Start.Y,

            EndX = entry.Model.End.X,
            EndY = entry.Model.End.Y
        });
    }

    return result;
}

internal void SelectFigure(int? id)
{
    if (_selectedVisual != null)
    {
        _selectedVisual.Effect = _originalEffect;
        _selectedVisual = null;
        _originalEffect = null;
    }
```

```
        if (id == null)
        {
            return;
        }

        for (int i = 0; i < _shapeCount; i++)
        {
            FigureEntry entry = _shapes[i]!;

            if (entry.Id == id)
            {
                _selectedVisual = entry.Visual;
                _originalEffect = entry.Visual.Effect;

                entry.Visual.Effect = new DropShadowEffect
                {
                    Color =
(Color)ColorConverter.ConvertFromString("#8dda19"),
                    ShadowDepth = 0,
                    BlurRadius = 20,
                    Opacity = 1
                };
```

```
            break;
        }
    }
}

internal bool DeleteFigure(int id)
{
    for (int i = 0; i < _shapeCount; i++)
    {
        FigureEntry entry = _shapes[i]!;

        if (entry.Id != id)
        {
            continue;
        }

        if (ReferenceEquals(_selectedVisual, entry.Visual))
        {
            SelectFigure(null);
        }
```

```
            GetCanvas().Children.Remove(entry.Visual);
            // зміщуємо всі елементи на один назад, щоб прибрати пустоту
            for (int j = i; j < _shapeCount - 1; j++)
            {
                _shapes[j] = _shapes[j + 1];
            }

            _shapeCount--;
            _shapes[_shapeCount] = null;

            ShapesChanged?.Invoke();

            return true;
        }

        return false;
    }

    internal void Clear()
    {
```

```
        CancelDrawing();
        SelectFigure(null);

        GetCanvas().Children.Clear();
        Array.Clear(_shapes, 0, _shapes.Length);

        _shapeCount = 0;
        _nextId = 1;

        ShapesChanged?.Invoke();
    }

    internal void LoadFigures(IReadOnlyList<FigureData> figures)
    {
        if (figures.Count > MaxShapes)
        {
            throw new InvalidDataException("У файлі забагато фігур.");
        }

        var prepared = new List<FigureEntry>();
        var usedIds = new HashSet<int>();
```

```
        foreach (FigureData data in figures)
        {
            if (data == null ||
                data.Id <= 0 ||
                data.Id >= int.MaxValue ||
                !usedIds.Add(data.Id))
            {
                throw new InvalidDataException("Некоректні ідентифікатори
фігур.");
            }

            if (!Enum.TryParse(data.Type, out DrawingTool tool) ||
                !Enum.IsDefined(tool))
            {
                throw new InvalidDataException("Невідомий тип геометричної
фігури.");
            }

            if (!double.IsFinite(data.StartX) ||
                !double.IsFinite(data.StartY) ||
                !double.IsFinite(data.EndX) ||
```

```
                !double.IsFinite(data.EndY))
            {
                throw new InvalidDataException("Некоректні координати
фігури.");
            }

            Shape shape = ShapeFactory.Create(
                tool,
                new Point(data.StartX, data.StartY),
                new Point(data.EndX, data.EndY)
            );

            prepared.Add(new FigureEntry(
                data.Id,
                tool,
                shape,
                shape.Show(false)
            ));
        }

        // змінюємо полотно лише після перевірки всіх фігур
        CancelDrawing();
```

```
        SelectFigure(null);



        Canvas canvas = GetCanvas();

        canvas.Children.Clear();



        Array.Clear(_shapes, 0, _shapes.Length);

        _shapeCount = 0;

        _nextId = 1;



        foreach (FigureEntry entry in prepared)

        {

            _shapes[_shapeCount++] = entry;

            canvas.Children.Add(entry.Visual);



            _nextId = Math.Max(_nextId, entry.Id + 1);

        }



        ShapesChanged?.Invoke();

    }

}
```

ShapeFactory.cs

```
using System.Windows;

using Lab4.Models;



namespace Lab4;



internal static class ShapeFactory

{

    public static Shape Create(

        DrawingTool tool,

        Point start,

        Point end)

    {

        return tool switch

        {

            DrawingTool.Point =>

                new PointShape(start),



            DrawingTool.Line =>

                new LineShape(start, end),
```

```
            DrawingTool.Rectangle =>

                new RectangleShape(start, end),



            DrawingTool.Ellipse =>

                new EllipseShape(start, end),



            DrawingTool.LineWithCircles =>

                new LineWithCirclesShape(start, end),



            DrawingTool.Cube =>

                new CubeShape(start, end),



            _ => throw new ArgumentOutOfRangeException(nameof(tool))

        };

    }

}
```

Shape.cs

```
using System.Windows;



namespace Lab4.Models;



```

```
// абстрактний клас (неможливо створити екземпляр, призначений лише для
наслідування)

// далі override у різних дочірніх класів - поліморфізм

internal abstract class Shape

{

    protected Shape(Point start, Point end)

    {

        Start = start;

        End = end;

    }



    public Point Start { get; }

    public Point End { get; }



    public abstract UIElement CreateVisual(bool isPreview);

    public UIElement Show(bool isPreview)

    {

        return CreateVisual(isPreview);

    }

}
```

PointShape.cs

```
using System.Windows;

using System.Windows.Controls;

using System.Windows.Shapes;

using System.Windows.Media;



namespace Lab4.Models;



internal class PointShape : Shape

{

    public PointShape(Point position) : base(position, position)

    {



    }



    public override UIElement CreateVisual(bool isPreview)

    {

        const double diameter = 6;



        var dot = new Ellipse

        {

            Width = diameter,
```

```
            Height = diameter,
            Fill = Brushes.Black,
            IsHitTestVisible = false
        };

        Canvas.SetLeft(dot, Start.X - diameter / 2);
        Canvas.SetTop(dot, Start.Y - diameter / 2);

        return dot;
    }
}
```

LineShape.cs

```
using System.Windows;
using System.Windows.Media;
using System.Windows.Shapes;

namespace Lab4.Models;

internal class LineShape : Shape
{
```

```
    public LineShape(Point start, Point end) : base(start, end)

    {

    }



    public override UIElement CreateVisual(bool isPreview)

    {

        var line = new Line

        {

            X1 = Start.X,

            Y1 = Start.Y,

            X2 = End.X,

            Y2 = End.Y,



            Stroke = isPreview ? Brushes.Blue : Brushes.Black,

            StrokeThickness = 2,



            StrokeDashArray = isPreview ? new DoubleCollection { 4, 3 } :
null,



            IsHitTestVisible = false

        };
```

```
        return line;

    }

}
```

RectangleShape.cs

```
using System.Windows;

using System.Windows.Controls;

using System.Windows.Media;

using System.Windows.Shapes;



namespace Lab4.Models;



internal class RectangleShape : Shape

{

    public RectangleShape(Point firstCorner, Point oppositeCorner)

    : base(firstCorner, oppositeCorner)

    {

    }



    public override UIElement CreateVisual(bool isPreview)

    {

        double left = Math.Min(Start.X, End.X);
```

```
        double top = Math.Min(Start.Y, End.Y);



        double width = Math.Abs(End.X - Start.X);

        double height = Math.Abs(End.Y - Start.Y);



        var rectangle = new Rectangle

        {

            Width = width,

            Height = height,



            Stroke = isPreview ? Brushes.Blue : Brushes.Black,

            StrokeThickness = 2,



            StrokeDashArray = isPreview ? new DoubleCollection { 4, 3 } :
null,



            Fill = isPreview ? null : Brushes.White,

            IsHitTestVisible = false

        };



        Canvas.SetLeft(rectangle, left);

        Canvas.SetTop(rectangle, top);
```

```
        return rectangle;
    }
}
```

EllipseShape.cs

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Shapes;

namespace Lab4.Models;

internal class EllipseShape : Shape
{
    public EllipseShape(Point center, Point corner)
    : base(center, corner)
    {
    }

    public override UIElement CreateVisual(bool isPreview)
    {
```

```
        double halfWidth = Math.Abs(End.X - Start.X);
        double halfHeight = Math.Abs(End.Y - Start.Y);

        double width = halfWidth * 2;
        double height = halfHeight * 2;

        double left = Start.X - halfWidth;
        double top = Start.Y - halfHeight;

        var ellipse = new Ellipse
        {
            Width = width,
            Height = height,

            Stroke = isPreview ? Brushes.Blue : Brushes.Black,
            StrokeThickness = 2,

            StrokeDashArray = isPreview ? new DoubleCollection { 4, 3 } :
null,

            Fill = null,
            IsHitTestVisible = false
```

```
        };



        Canvas.SetLeft(ellipse, left);

        Canvas.SetTop(ellipse, top);



        return ellipse;

    }

}
```

LineWithCirclesShape.cs

```
using System.Windows;

using System.Windows.Controls;

using System.Windows.Media;

using System.Windows.Shapes;



namespace Lab4.Models;



internal sealed class LineWithCirclesShape : Shape

{

    private const double Radius = 7;

    public LineWithCirclesShape(Point start, Point end)

        : base(start, end)
```

```
    {

    }


    public override UIElement CreateVisual(bool isPreview)

    {

        Canvas group = new Canvas

        {

            Width = Math.Max(1, Math.Max(Start.X, End.X) + Radius + 2),

            Height = Math.Max(1, Math.Max(Start.Y, End.Y) + Radius + 2),

            IsHitTestVisible = false

        };



        LineShape line = new LineShape(Start, End);

        group.Children.Add(line.Show(isPreview));



        AddCircle(group, Start, isPreview);

        AddCircle(group, End, isPreview);



        return group;

    }
```

```
    private static void AddCircle(

        Canvas group,

        Point center,

        bool isPreview)

    {

        Point corner = new Point(

            center.X + Radius,

            center.Y + Radius

        );



        EllipseShape ellipse = new EllipseShape(center, corner);

        Ellipse visual = (Ellipse)ellipse.Show(isPreview);



        visual.Fill = isPreview ? null : Brushes.LightBlue;



        group.Children.Add(visual);

    }

}
```

CubeShape.cs

```
using System.Windows;

using System.Windows.Controls;
```

```
using System.Windows.Shapes;



namespace Lab4.Models;



internal sealed class CubeShape : Shape

{

    public CubeShape(Point start, Point end)

        : base(start, end)

    {

    }



    public override UIElement CreateVisual(bool isPreview)

    {

        double width = Math.Abs(End.X - Start.X);

        double height = Math.Abs(End.Y - Start.Y);



        double side = Math.Max(width, height);



        double left = Math.Min(Start.X, End.X);

        double top = Math.Min(Start.Y, End.Y);
```

```
        double depth = side * 0.25;



        Point[] front =

        {

            new Point(left, top),

            new Point(left + side, top),

            new Point(left + side, top + side),

            new Point(left, top + side)

        };



        Point[] back = new Point[4];



        for (int i = 0; i < 4; i++)

        {

            back[i] = new Point(

                front[i].X + depth,

                front[i].Y - depth

            );

        }



        Canvas group = new Canvas
```

```
        {
            Width = Math.Max(1, left + side + depth + 2),
            Height = Math.Max(1, top + side + 2),
            IsHitTestVisible = false
        };

        RectangleShape backRectangle = new RectangleShape(back[0],
back[2]);
        Rectangle backVisual = (Rectangle)backRectangle.Show(isPreview);
        backVisual.Fill = null;

        group.Children.Add(backVisual);

        //  з'єднувальні лінії
        for (int i = 0; i < 4; i++)
        {
            LineShape edge = new LineShape(
                front[i],
                back[i]
            );

            group.Children.Add(edge.Show(isPreview));
```

```
        }



        RectangleShape frontRectangle = new RectangleShape(front[0],
front[2]);

        Rectangle frontVisual = (Rectangle)frontRectangle.Show(isPreview);

        frontVisual.Fill = null;



        group.Children.Add(frontVisual);



        return group;

    }

}
```

FigureData.cs

```
namespace Lab5;



internal class FigureData

{

    public int Id { get; set; }



    public string Type { get; set; } = "";
```

```
    public double StartX { get; set; }

    public double StartY { get; set; }



    public double EndX { get; set; }

    public double EndY { get; set; }

}
```

FigureStorage.cs

```
using System.Collections.Generic;

using System.IO;

using System.Text.Json;



namespace Lab5;



internal static class FigureStorage

{

    internal static void Save(string path, IReadOnlyList<FigureData>
figures)

    {

        var options = new JsonSerializerOptions

        {

            WriteIndented = true
```

```
        };

        string json = JsonSerializer.Serialize(
            figures,
            options
        );

        File.WriteAllText(path, json);
    }

    internal static List<FigureData> Load(string path)
    {
        string json = File.ReadAllText(path);

        return JsonSerializer.Deserialize<List<FigureData>>(json)
            ?? throw new InvalidDataException(
                "Файл не містить коректного списку фігур.");
    }
}
```

MyTableWindow.xaml.cs

```
using System;

using System.Collections.Generic;

using System.Collections.ObjectModel;

using System.Linq;

using System.Windows;

using System.Windows.Controls;



namespace Lab5;



public class TableRow

{

    public int Id { get; init; }



    public string Type { get; init; } = "";



    public double X1 { get; init; }

    public double Y1 { get; init; }



    public double X2 { get; init; }

    public double Y2 { get; init; }

}
```

```
public partial class MyTableWindow : Window

{

    private readonly ObservableCollection<TableRow> _rows = new();

    private bool _updating;

    public event Action<int?>? RowSelected;

    public event Action<int>? DeleteRequested;




    public MyTableWindow()

    {

        InitializeComponent();

        objectsGrid.ItemsSource = _rows;

    }




    public void SetRows(IEnumerable<TableRow> rows)

    {

        int? selectedId = (objectsGrid.SelectedItem as TableRow)?.Id;

        _updating = true;
```

```
try
{
    _rows.Clear();

    foreach (TableRow row in rows)
    {
        _rows.Add(row);
    }

    if (selectedId != null &&
        deleteModeCheckBox.IsChecked != true)
    {
        objectsGrid.SelectedItem = _rows
            .FirstOrDefault(row => row.Id == selectedId);
    }
}
finally
{
    _updating = false;
}
```

```
    }



    private void ObjectsGrid_SelectionChanged(object sender,
SelectionChangedEventArgs e)
    {
        if (_updating)
        {
            return;
        }

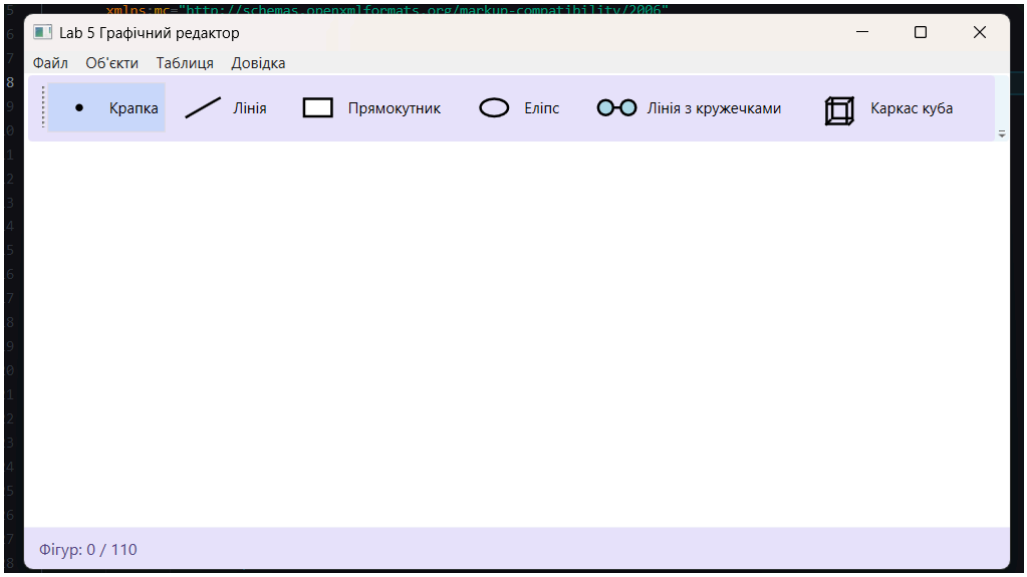
        if (objectsGrid.SelectedItem is not TableRow row)
        {
            RowSelected?.Invoke(null);
            return;
        }

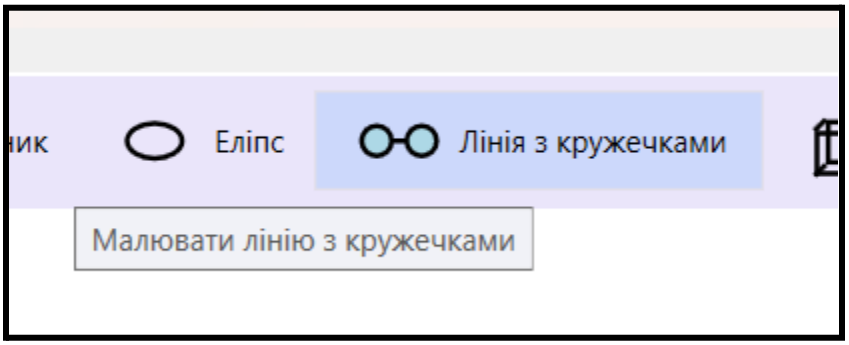
        if (deleteModeCheckBox.IsChecked == true)
        {
            DeleteRequested?.Invoke(row.Id);
            objectsGrid.SelectedItem = null;
        }
```

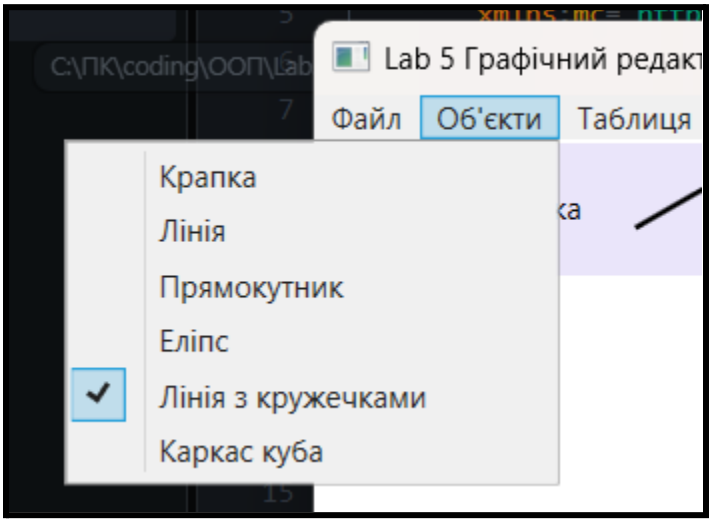
```
        else
        {
            RowSelected?.Invoke(row.Id);
        }
    }

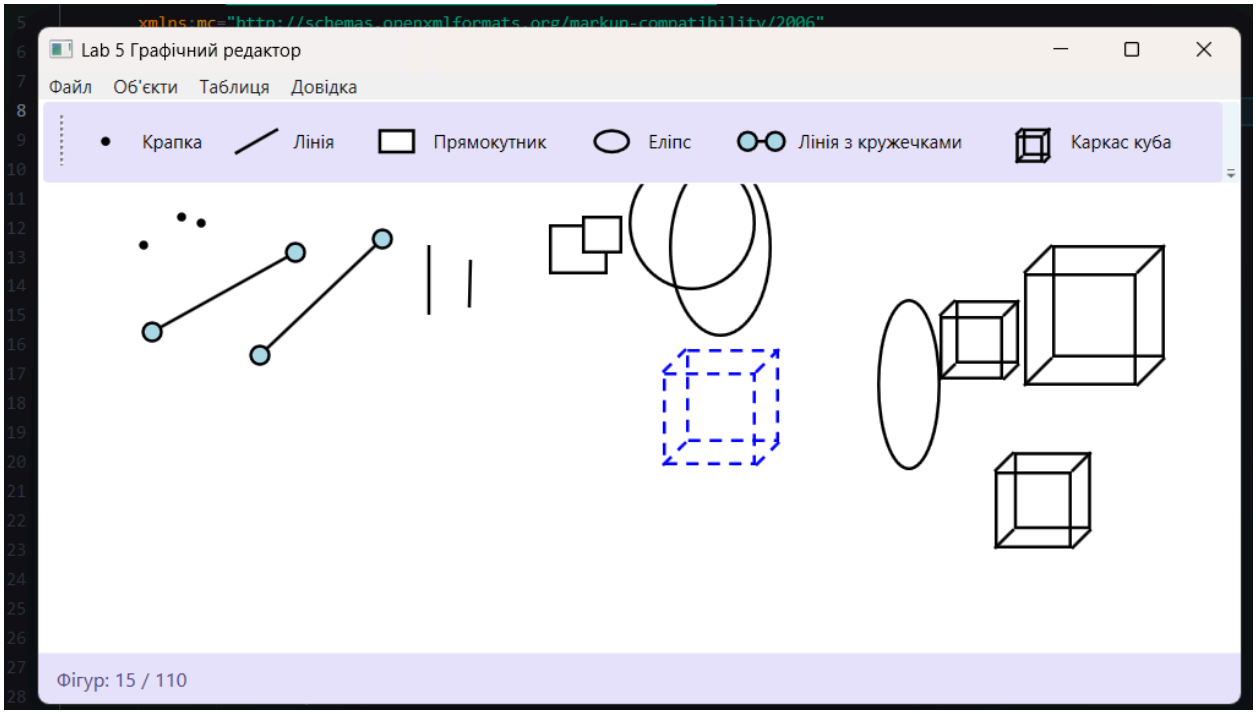

    private void DeleteMode_Checked(object sender, RoutedEventArgs e)
    {
        objectsGrid.SelectedItem = null;
        RowSelected?.Invoke(null);
    }
}
```

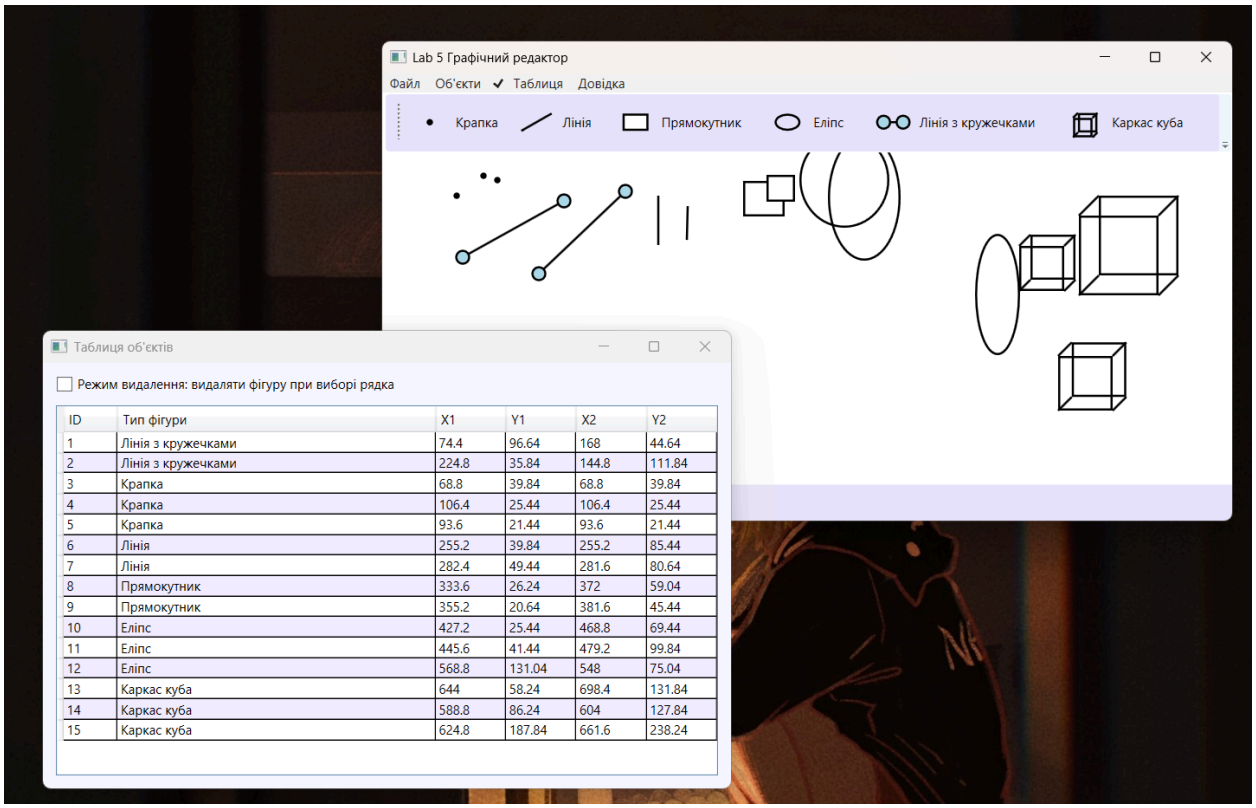
Скріншоти


Lab 5 Графічний редактор
Файл
Об'єкти
Таблиця
Довідка
Крапка
Лінія
Прямокутник
Еліпс
Лінія з кружечками
Каркас куба
Фігур: 0 / 110

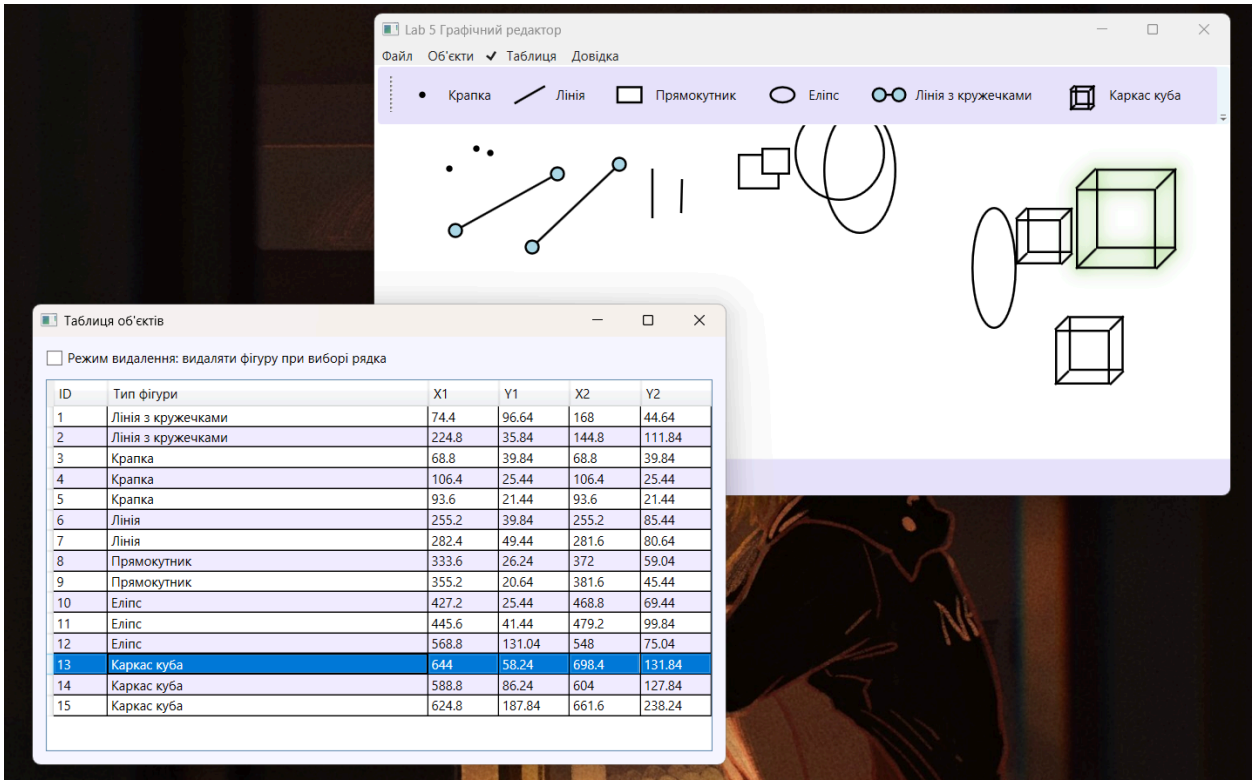

Еліпс
Лінія з кружечками
Малювати лінію з кружечками


Lab 5 Графічний редакт
Файл
Об'єкти
Таблиця
Крапка
Лінія
Прямокутник
Еліпс
Лінія з кружечками
Каркас куба

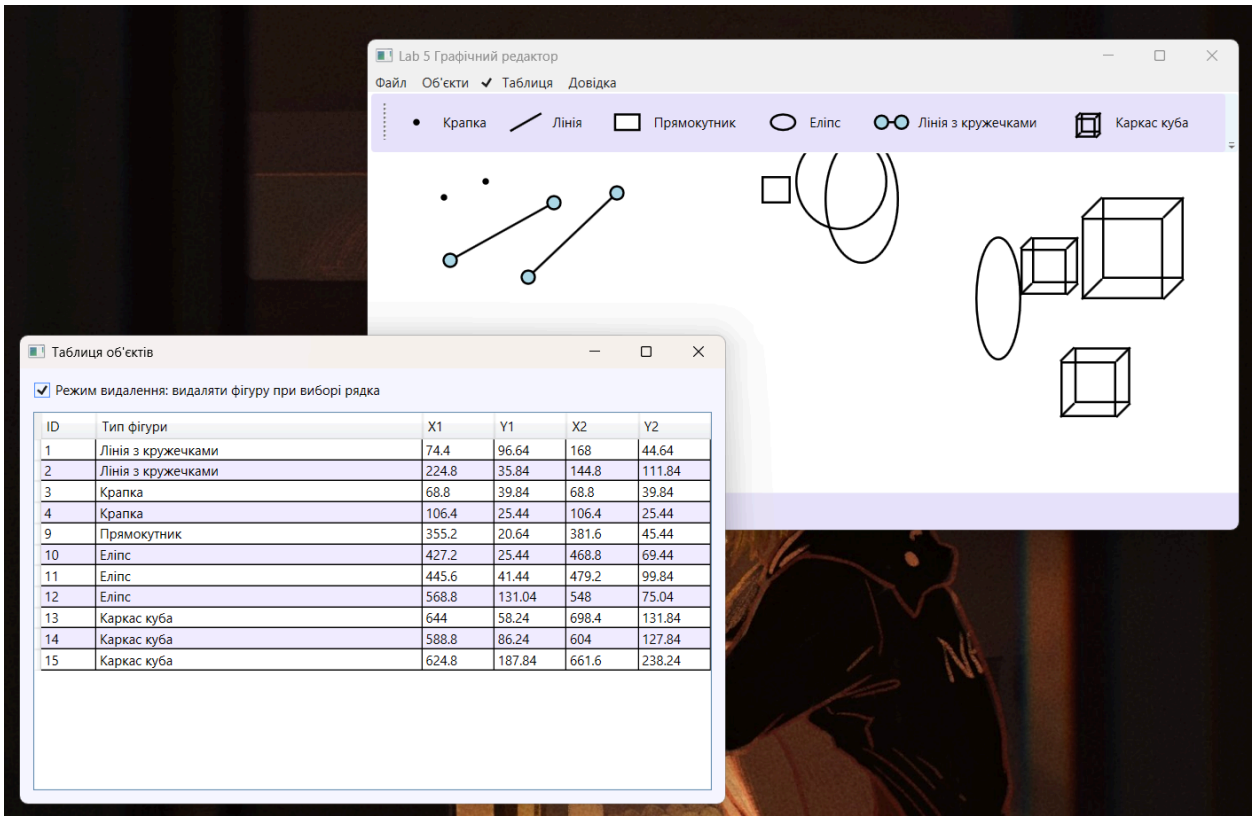




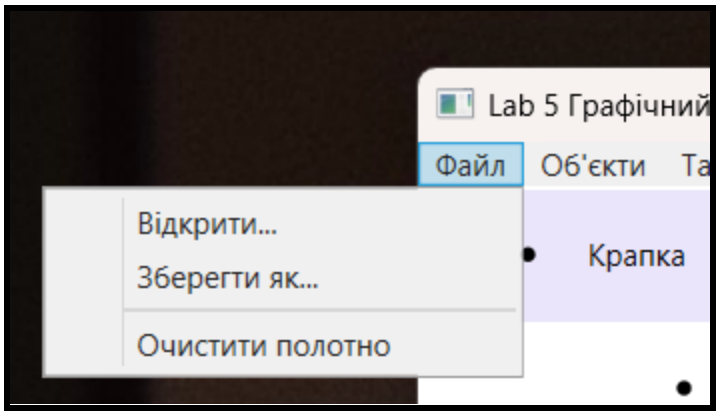
ID	Тип фігури	X1	Y1	X2	Y2
1	Лінія з кружечками	74.4	96.64	168	44.64
2	Лінія з кружечками	224.8	35.84	144.8	111.84
3	Крапка	68.8	39.84	68.8	39.84
4	Крапка	106.4	25.44	106.4	25.44
5	Крапка	93.6	21.44	93.6	21.44
6	Лінія	255.2	39.84	255.2	85.44
7	Лінія	282.4	49.44	281.6	80.64
8	Прямокутник	333.6	26.24	372	59.04
9	Прямокутник	355.2	20.64	381.6	45.44
10	Еліпс	427.2	25.44	468.8	69.44
11	Еліпс	445.6	41.44	479.2	99.84
12	Еліпс	568.8	131.04	548	75.04
13	Каркас куба	644	58.24	698.4	131.84
14	Каркас куба	588.8	86.24	604	127.84
15	Каркас куба	624.8	187.84	661.6	238.24



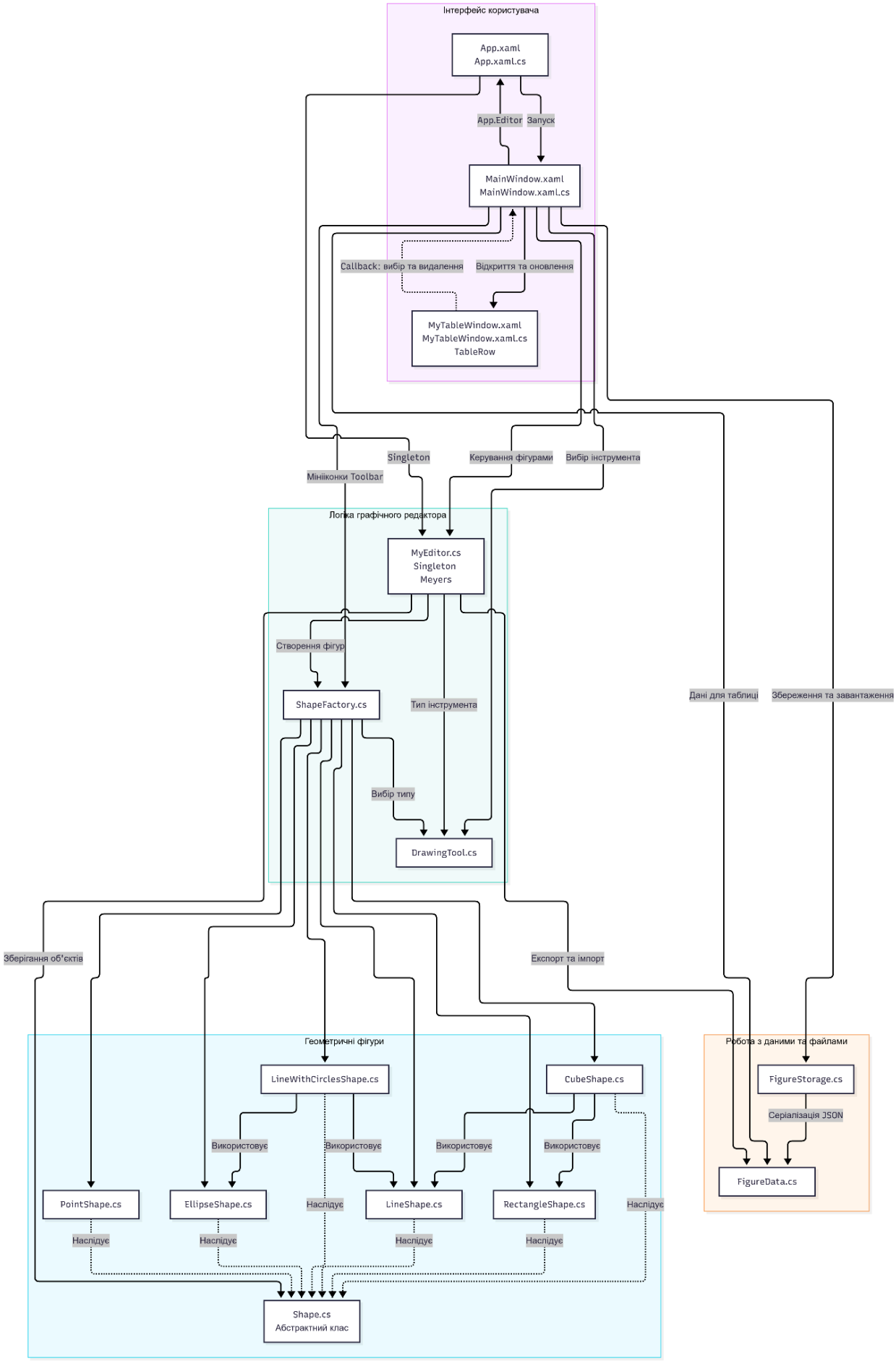
ID	Тип фігури	X1	Y1	X2	Y2
1	Лінія з кружечками	74.4	96.64	168	44.64
2	Лінія з кружечками	224.8	35.84	144.8	111.84
3	Крапка	68.8	39.84	68.8	39.84
4	Крапка	106.4	25.44	106.4	25.44
5	Крапка	93.6	21.44	93.6	21.44
6	Лінія	255.2	39.84	255.2	85.44
7	Лінія	282.4	49.44	281.6	80.64
8	Прямокутник	333.6	26.24	372	59.04
9	Прямокутник	355.2	20.64	381.6	45.44
10	Еліпс	427.2	25.44	468.8	69.44
11	Еліпс	445.6	41.44	479.2	99.84
12	Еліпс	568.8	131.04	548	75.04
13	Каркас куба	644	58.24	698.4	131.84
14	Каркас куба	588.8	86.24	604	127.84
15	Каркас куба	624.8	187.84	661.6	238.24



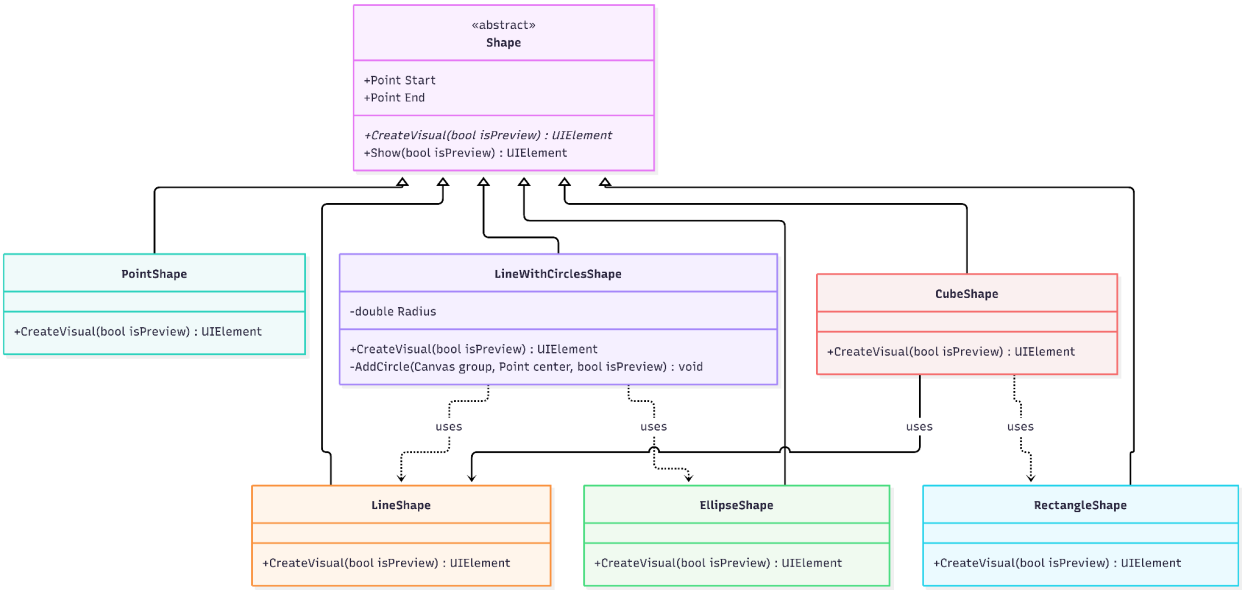
ID	Тип фігури	X1	Y1	X2	Y2
1	Лінія з кружечками	74.4	96.64	168	44.64
2	Лінія з кружечками	224.8	35.84	144.8	111.84
3	Крапка	68.8	39.84	68.8	39.84
4	Крапка	106.4	25.44	106.4	25.44
9	Прямокутник	355.2	20.64	381.6	45.44
10	Еліпс	427.2	25.44	468.8	69.44
11	Еліпс	445.6	41.44	479.2	99.84
12	Еліпс	568.8	131.04	548	75.04
13	Каркас куба	644	58.24	698.4	131.84
14	Каркас куба	588.8	86.24	604	127.84
15	Каркас куба	624.8	187.84	661.6	238.24



Діаграма залежностей файлів та модулів


Інтерфейс користувача
App.xaml
App.xaml.cs
App.Editor
Запуск
MainWindow.xaml
MainWindow.xaml.cs
Callback: вибір та видалення
Відкриття та оновлення
MyTableWindow.xaml
MyTableWindow.xaml.cs
TableRow
Singleton
Керування фігурами
Вибір інструмента
Мініконки Toolbar
Логіка графічного редактора
MyEditor.cs
Singleton
Meyers
Створення фігур
ShapeFactory.cs
Тип інструмента
Вибір типу
DrawingTool.cs
Дані для таблиці
Збереження та завантаження
Зберігання об'єктів
Експорт та імпорт
Геометричні фігури
LineWithCirclesShape.cs
CubeShape.cs
Використовує
Використовує
Використовує
Використовує
PointShape.cs
EllipseShape.cs
Наслідує
LineShape.cs
RectangleShape.cs
Наслідує
Наслідує
Наслідує
Наслідує
Наслідує
Shape.cs
Абстрактний клас
Робота з даними та файлами
FigureStorage.cs
Серіалізація JSON
FigureData.cs

Діаграма успадкування класів



Приклад текстового файлу множини об’єктів


```
[
  {
    "Id": 1,
    "Type": "Ellipse",
    "StartX": 332.8,
    "StartY": 138.24,
    "EndX": 366.40000000000003,
    "EndY": 192.64000000000001
  },
  {
```

```
  "Id": 6,
  "Type": "Ellipse",
  "StartX": 332,
  "StartY": 54.239999999999995,
  "EndX": 366.40000000000003,
  "EndY": 87.84
},
{
  "Id": 7,
  "Type": "LineWithCircles",
  "StartX": 307.20000000000005,
  "StartY": 104.64000000000001,
  "EndX": 216.8,
  "EndY": 133.44
},
{
  "Id": 9,
  "Type": "LineWithCircles",
  "StartX": 426.40000000000003,
  "StartY": 134.24,
  "EndX": 466.40000000000003,
  "EndY": 62.24000000000001
},
{
```

```
  "Id": 11,
  "Type": "LineWithCircles",
  "StartX": 311.20000000000005,
  "StartY": 276.64,
  "EndX": 270.40000000000003,
  "EndY": 273.44000000000005
},
{
  "Id": 12,
  "Type": "LineWithCircles",
  "StartX": 341.6,
  "StartY": 197.44000000000003,
  "EndX": 345.6,
  "EndY": 291.84000000000003
},
{
  "Id": 13,
  "Type": "LineWithCircles",
  "StartX": 346.40000000000003,
  "StartY": 292.64,
  "EndX": 360.8,
  "EndY": 291.03999999999996
},
{
```

```
  "Id": 14,
  "Type": "Point",
  "StartX": 311.20000000000005,
  "StartY": 46.239999999999995,
  "EndX": 311.20000000000005,
  "EndY": 46.239999999999995
},
{
  "Id": 15,
  "Type": "Point",
  "StartX": 348.8,
  "StartY": 46.239999999999995,
  "EndX": 348.8,
  "EndY": 46.239999999999995
},
{
  "Id": 16,
  "Type": "Line",
  "StartX": 315.20000000000005,
  "StartY": 63.03999999999999,
  "EndX": 320.8,
  "EndY": 68.63999999999999
},
{
```

```
  "Id": 17,
  "Type": "Line",
  "StartX": 321.6,
  "StartY": 69.44,
  "EndX": 329.6,
  "EndY": 62.24000000000001
},
{
  "Id": 18,
  "Type": "Line",
  "StartX": 329.6,
  "StartY": 62.24000000000001,
  "EndX": 336,
  "EndY": 69.44
},
{
  "Id": 19,
  "Type": "Line",
  "StartX": 336,
  "StartY": 69.44,
  "EndX": 342.40000000000003,
  "EndY": 64.63999999999999
},
{
```

```
  "Id": 20,
  "Type": "Line",
  "StartX": 310.40000000000003,
  "StartY": 27.039999999999992,
  "EndX": 253.60000000000002,
  "EndY": 17.439999999999998
},
{
  "Id": 21,
  "Type": "Line",
  "StartX": 253.60000000000002,
  "StartY": 17.439999999999998,
  "EndX": 296.8,
  "EndY": 61.44
},
{
  "Id": 22,
  "Type": "Line",
  "StartX": 357.6,
  "StartY": 31.840000000000003,
  "EndX": 408,
  "EndY": 20.64
},
{
```

```
  "Id": 23,
  "Type": "Line",
  "StartX": 408.8,
  "StartY": 20.64,
  "EndX": 363.20000000000005,
  "EndY": 67.03999999999999
},
{
  "Id": 24,
  "Type": "Line",
  "StartX": 307.20000000000005,
  "StartY": 64.63999999999999,
  "EndX": 252.8,
  "EndY": 62.24000000000001
},
{
  "Id": 25,
  "Type": "Line",
  "StartX": 308,
  "StartY": 70.24000000000001,
  "EndX": 269.6,
  "EndY": 81.44
},
{
```

```
  "Id": 26,
  "Type": "Line",
  "StartX": 309.6,
  "StartY": 73.44,
  "EndX": 256,
  "EndY": 99.84
},
{
  "Id": 27,
  "Type": "Line",
  "StartX": 352,
  "StartY": 70.24000000000001,
  "EndX": 417.6,
  "EndY": 55.84
},
{
  "Id": 28,
  "Type": "Line",
  "StartX": 354.40000000000003,
  "StartY": 75.03999999999999,
  "EndX": 392.8,
  "EndY": 79.03999999999999
},
{
```

```
  "Id": 29,
  "Type": "Line",
  "StartX": 352.8,
  "StartY": 77.44,
  "EndX": 380,
  "EndY": 95.03999999999999
},
{
  "Id": 30,
  "Type": "Point",
  "StartX": 328,
  "StartY": 58.24000000000001,
  "EndX": 328,
  "EndY": 58.24000000000001
},
{
  "Id": 31,
  "Type": "LineWithCircles",
  "StartX": 356.8,
  "StartY": 99.84,
  "EndX": 423.20000000000005,
  "EndY": 138.24
},
{
```

```
  "Id": 32,
  "Type": "LineWithCircles",
  "StartX": 316.8,
  "StartY": 188.64000000000001,
  "EndX": 315.20000000000005,
  "EndY": 274.24
},
{
  "Id": 34,
  "Type": "Line",
  "StartX": 521.6,
  "StartY": 134.24,
  "EndX": 560.8000000000001,
  "EndY": 154.24
},
{
  "Id": 35,
  "Type": "Line",
  "StartX": 520,
  "StartY": 136.64000000000001,
  "EndX": 479.20000000000005,
  "EndY": 196.64000000000001
},
{
```

```
  "Id": 36,
  "Type": "Line",
  "StartX": 479.20000000000005,
  "StartY": 196.64000000000001,
  "EndX": 524,
  "EndY": 227.84
},
{
  "Id": 37,
  "Type": "Line",
  "StartX": 524,
  "StartY": 227.84,
  "EndX": 542.4,
  "EndY": 219.04
},
{
  "Id": 38,
  "Type": "Line",
  "StartX": 591.2,
  "StartY": 160.64000000000001,
  "EndX": 568,
  "EndY": 229.44000000000003
},
{
```

```
  "Id": 39,
  "Type": "Line",
  "StartX": 617.6,
  "StartY": 155.04,
  "EndX": 589.6,
  "EndY": 230.23999999999998
},
{
  "Id": 40,
  "Type": "Line",
  "StartX": 560,
  "StartY": 195.84,
  "EndX": 640,
  "EndY": 168.64000000000001
},
{
  "Id": 41,
  "Type": "Line",
  "StartX": 559.2,
  "StartY": 207.84,
  "EndX": 642.4000000000001,
  "EndY": 183.04
},
{
```

```
  "Id": 42,
  "Type": "LineWithCircles",
  "StartX": 98.4,
  "StartY": 227.04,
  "EndX": 39.2,
  "EndY": 167.84
},
{
  "Id": 43,
  "Type": "LineWithCircles",
  "StartX": 39.2,
  "StartY": 167.84,
  "EndX": 92.80000000000001,
  "EndY": 124.64000000000001
},
{
  "Id": 44,
  "Type": "LineWithCircles",
  "StartX": 93.60000000000001,
  "StartY": 125.44,
  "EndX": 110.4,
  "EndY": 164.64000000000001
},
{
```

```
  "Id": 45,
  "Type": "LineWithCircles",
  "StartX": 110.4,
  "StartY": 164.64000000000001,
  "EndX": 158.4,
  "EndY": 139.04
},
{
  "Id": 46,
  "Type": "LineWithCircles",
  "StartX": 158.4,
  "StartY": 139.04,
  "EndX": 178.4,
  "EndY": 183.84
},
{
  "Id": 47,
  "Type": "LineWithCircles",
  "StartX": 178.4,
  "StartY": 183.84,
  "EndX": 104.80000000000001,
  "EndY": 231.84
},
{
```

```
  "Id": 48,
  "Type": "Point",
  "StartX": 93.60000000000001,
  "StartY": 188.64000000000001,
  "EndX": 93.60000000000001,
  "EndY": 188.64000000000001
},
{
  "Id": 49,
  "Type": "Point",
  "StartX": 108,
  "StartY": 187.04,
  "EndX": 108,
  "EndY": 187.04
},
{
  "Id": 50,
  "Type": "Line",
  "StartX": 86.4,
  "StartY": 201.44000000000003,
  "EndX": 108.80000000000001,
  "EndY": 206.23999999999998
},
{
```

```
    "Id": 51,
    "Type": "Line",
    "StartX": 108.80000000000001,
    "StartY": 206.23999999999998,
    "EndX": 131.20000000000002,
    "EndY": 192.64000000000001
  }
]
```


Висновки:

У ході лабораторної роботи було розроблено багатовіконний інтерфейс графічного редактора об’єктів із використанням принципів об’єктно-орієнтованого програмування. Реалізовано Singleton Меєрса, немодальне вікно таблиці та взаємодію між модулями за допомогою подій. Додатково реалізовано виділення та видалення фігур через таблицю, а також збереження й завантаження об’єктів у форматі JSON. У результаті було закріплено навички роботи з багатовіконними застосунками WPF, подіями та організацією взаємодії між класами.